PYTHON PROGRAMS

Algorithms, Flowcharts, Pseudocode, Test Cases and Output Screenshots

Prepared in simple English based on the provided PyCharm screenshots.

Note: The flowcharts are written as easy arrow steps so they can be copied into a notebook or redrawn using standard flowchart symbols.

1. Shopping Bill Calculator

Aim: To calculate the total cost of three items using their prices and quantities.

Algorithm

1. Start the program.
2. Set total to 0.
3. Repeat for three items: read the price and quantity.
4. Check that price and quantity are not negative. If either is negative, display an error and stop.
5. Multiply price by quantity and add the result to total.
6. Display the total shopping bill to two decimal places.
7. Stop.

Flowchart (text form)

Start → Set total = 0 → Enter price and quantity for each of 3 items → Are values non-negative? → No: show error and stop / Yes: add price × quantity → More items? → Yes: repeat input / No: display total → Stop

Pseudocode

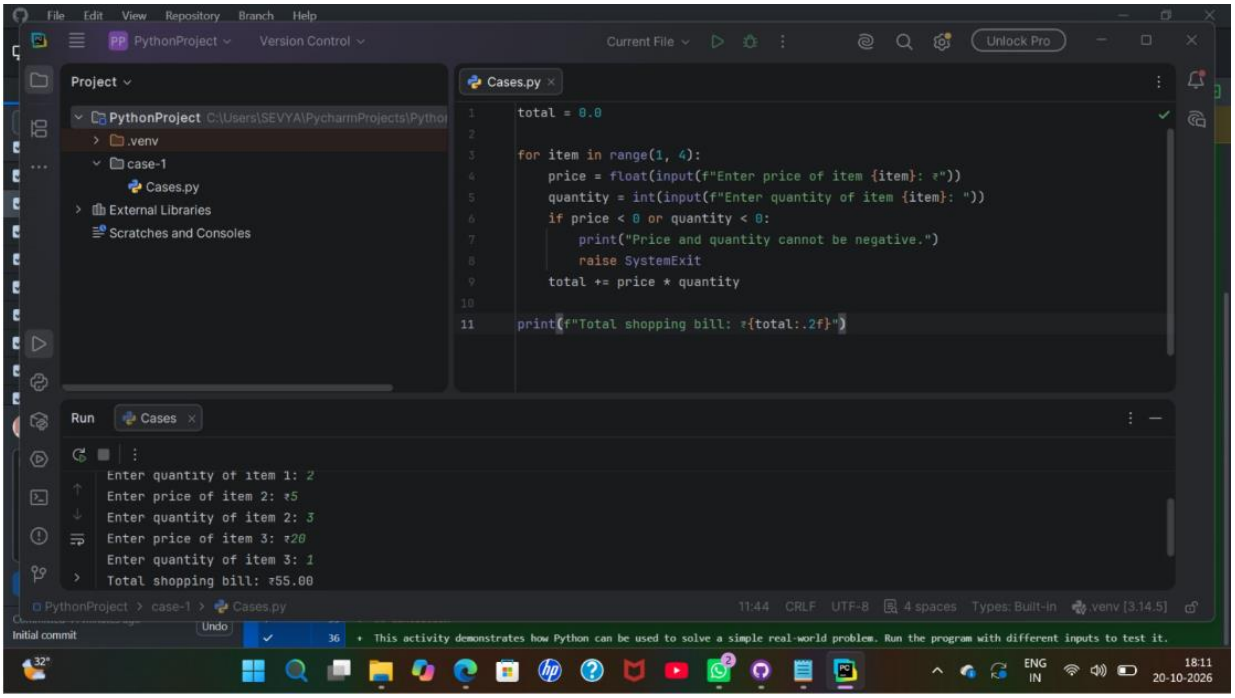
```
START
total ← 0
FOR item ← 1 TO 3
    INPUT price, quantity
    IF price < 0 OR quantity < 0 THEN
        DISPLAY error message
        STOP
    END IF
    total ← total + (price × quantity)
END FOR
DISPLAY total
STOP
```

Test Cases

Test	Input	Expected result
3 items with valid prices and quantities	Price/quantity values are entered for all three items	Total bill is displayed with two decimal places
A negative price or quantity	Enter a negative value	Error message is displayed and program stops

Output Screenshot

The screenshot shows the total shopping bill as ₹55.00.


File Edit View Repository Branch Help
PythonProject
Version Control
Current File
Unlock Pro
Project
PythonProject C:\Users\SEVYA\PycharmProjects\Pytho
.venv
case-1
Cases.py
External Libraries
Scratches and Consoles
Cases.py
total = 0.0
for item in range(1, 4):
price = float(input(f"Enter price of item {item}: ₹"))
quantity = int(input(f"Enter quantity of item {item}: "))
if price < 0 or quantity < 0:
print("Price and quantity cannot be negative.")
raise SystemExit
total += price * quantity
print(f"Total shopping bill: ₹{total:.2f}")
Run
Cases
Enter quantity of item 1: 2
Enter price of item 2: ₹5
Enter quantity of item 2: 3
Enter price of item 3: ₹20
Enter quantity of item 3: 1
Total shopping bill: ₹55.00
PythonProject > case-1 > Cases.py
11:44 CRLF UTF-8 4 spaces Types: Built-in .venv [3.14.5]
Initial commit
Undo
36
This activity demonstrates how Python can be used to solve a simple real-world problem. Run the program with different inputs to test it.
32°
ENG
IN
18:11
20-10-2026

2. Retail Billing System

Aim: To prepare a bill for any number of items and calculate the grand total.

Algorithm

8. Start and display the heading RETAIL BILLING.
9. Ask how many items are to be entered.
10. Set grand_total to 0.
11. For each item, read its name, quantity and price per item.
12. Calculate subtotal = quantity × price.
13. Add the subtotal to grand_total and display the item details.
14. After all items are entered, display the grand total.
15. Stop.

Flowchart (text form)

Start → Enter number of items → Set grand total = 0 → Enter item name, quantity and price → Calculate subtotal → Add subtotal to grand total → More items? → Yes: repeat / No: display grand total → Stop

Pseudocode

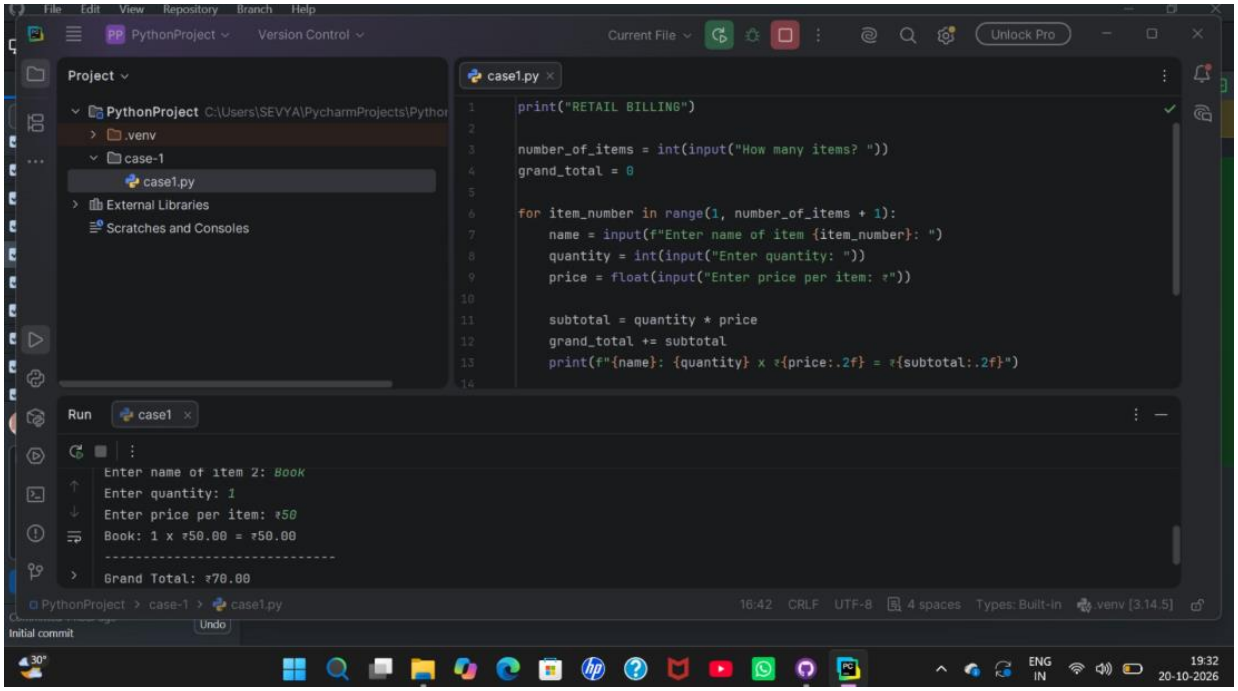
```
START
DISPLAY "RETAIL BILLING"
INPUT number_of_items
grand_total ← 0
FOR each item from 1 to number_of_items
    INPUT name, quantity, price
    subtotal ← quantity × price
    grand_total ← grand_total + subtotal
    DISPLAY name, quantity, price, subtotal
END FOR
DISPLAY grand_total
STOP
```

Test Cases

Test	Input	Expected result
One book, quantity 1, price ₹50	Book; 1; ₹50	Book subtotal = ₹50.00; grand total includes all entered items
Two or more items	Enter names, quantities and prices	Each subtotal and final grand total are displayed

Output Screenshot

The visible output includes Book: 1 × ₹50.00 = ₹50.00 and Grand Total: ₹70.00.


File Edit View Repository Branch Help
PythonProject
Version Control
Current File
Unlock Pro
Project
PythonProject C:\Users\SEVYA\PycharmProjects\Pytho
.venv
case-1
case1.py
External Libraries
Scratches and Consoles
case1.py
print("RETAIL BILLING")
number_of_items = int(input("How many items? "))
grand_total = 0
for item_number in range(1, number_of_items + 1):
name = input(f"Enter name of item {item_number}: ")
quantity = int(input("Enter quantity: "))
price = float(input("Enter price per item: ₹"))
subtotal = quantity * price
grand_total += subtotal
print(f"{name}: {quantity} x ₹{price:.2f} = ₹{subtotal:.2f}")
Run
case1
Enter name of item 2: Book
Enter quantity: 1
Enter price per item: ₹50
Book: 1 x ₹50.00 = ₹50.00
Grand Total: ₹70.00
PythonProject > case-1 > case1.py
16:42 CRLF UTF-8 4 spaces Types: Built-in .venv [3.14.5]
Initial commit
Undo
30°
ENG
IN
19:32
20-10-2026

3. Exam Result Calculator

Aim: To calculate average marks, assign a grade and decide whether the student passes.

Algorithm

16. Start and ask for the number of subjects.
17. Set total marks to 0 and assume the student passes.
18. For every subject, read marks out of 100.
19. If marks are below 0 or above 100, display an error and stop.
20. Add the marks to total. If any mark is below 35, set the pass flag to false.
21. Calculate average = total marks ÷ number of subjects.
22. Assign the grade using the program's grade conditions.
23. Display average, grade and pass/fail result.
24. Stop.

Flowchart (text form)

Start → Enter number of subjects → Enter each mark → Marks between 0 and 100? → No: show error and stop / Yes: add marks and check pass mark → All subjects entered? → Calculate average and grade → Display result → Stop

Pseudocode

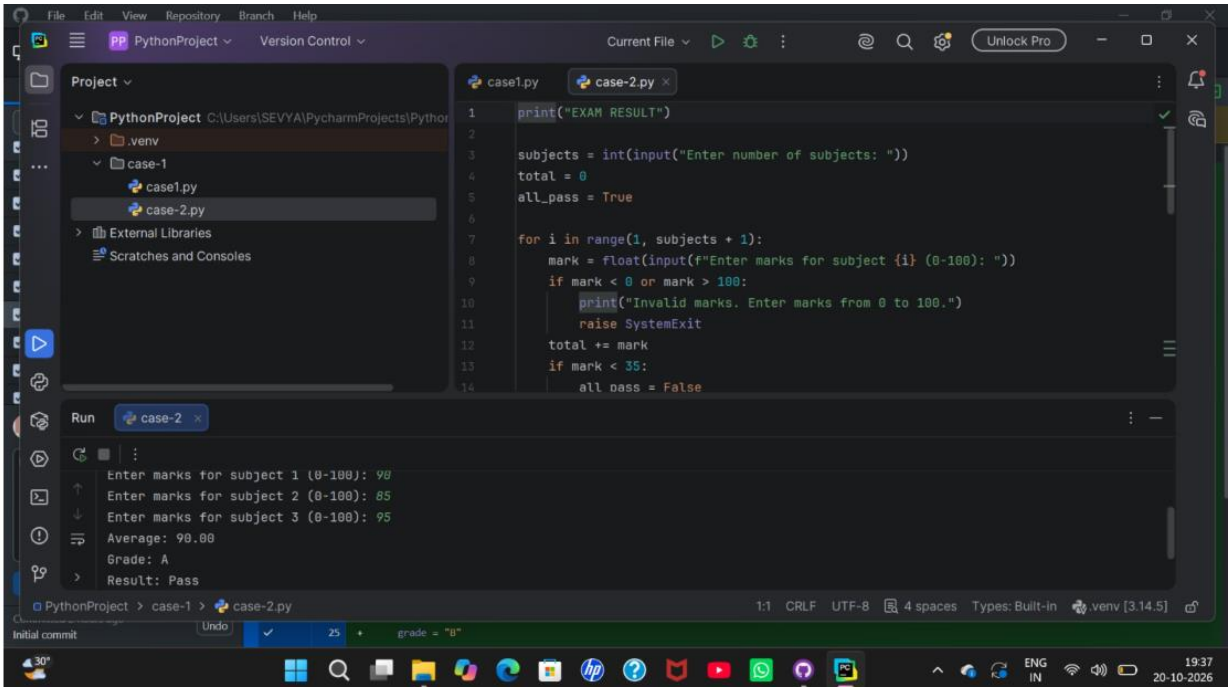
```
START
INPUT subjects
total ← 0; all_pass ← TRUE
FOR i ← 1 TO subjects
    INPUT mark
    IF mark < 0 OR mark > 100 THEN
        DISPLAY invalid marks message
        STOP
    END IF
    total ← total + mark
    IF mark < 35 THEN all_pass ← FALSE
END FOR
average ← total ÷ subjects
ASSIGN grade using grade rules
DISPLAY average, grade, pass/fail
STOP
```

Test Cases

Test	Input	Expected result
Marks 90, 85, 95	Three valid marks	Average = 90.00, Grade A, Result Pass
A mark below 35	For example, 90, 30, 95	Result should be Fail because one subject is below 35
Invalid mark	Enter 105	Invalid-marks message appears and program stops

Output Screenshot

The screenshot shows average 90.00, Grade A and Result: Pass.


File Edit View Repository Branch Help
PythonProject
Version Control
Current File
Unlock Pro
Project
PythonProject C:\Users\SEVYA\PycharmProjects\Pythor
.venv
case-1
case1.py
case-2.py
External Libraries
Scratches and Consoles
case1.py
case-2.py
print("EXAM RESULT")
subjects = int(input("Enter number of subjects: "))
total = 0
all_pass = True
for i in range(1, subjects + 1):
mark = float(input(f"Enter marks for subject {i} (0-100): "))
if mark < 0 or mark > 100:
print("Invalid marks. Enter marks from 0 to 100.")
raise SystemExit
total += mark
if mark < 35:
all_pass = False
Run
case-2
Enter marks for subject 1 (0-100): 90
Enter marks for subject 2 (0-100): 85
Enter marks for subject 3 (0-100): 95
Average: 90.00
Grade: A
Result: Pass
PythonProject > case-1 > case-2.py
1:1 CRLF UTF-8 4 spaces Types: Built-in .venv [3.14.5]
Initial commit
Undo
grade = "B"
ENG
IN
19:37
20-10-2026

4. Attendance Recovery Calculator

Aim: To find how many extra classes a student must attend to reach a target attendance percentage.

Algorithm

25. Start and input classes attended, total classes and target percentage.
26. Check that total classes is positive, attended classes are not negative or greater than total, and target is between 0 and 100.
27. Calculate current attendance percentage = attended ÷ total × 100.
28. Set extra classes to 0.
29. While the current percentage is below the target, increase attended classes and total classes by one and increase extra classes by one.
30. Display current attendance, extra classes needed and attendance after recovery.
31. Stop.

Flowchart (text form)

Start → Input attended classes, total classes and target % → Are values valid? → No: show error / Yes: calculate current % → Is current % below target? → Yes: add one attended class and one total class; count extra class; repeat → No: display result → Stop

Pseudocode

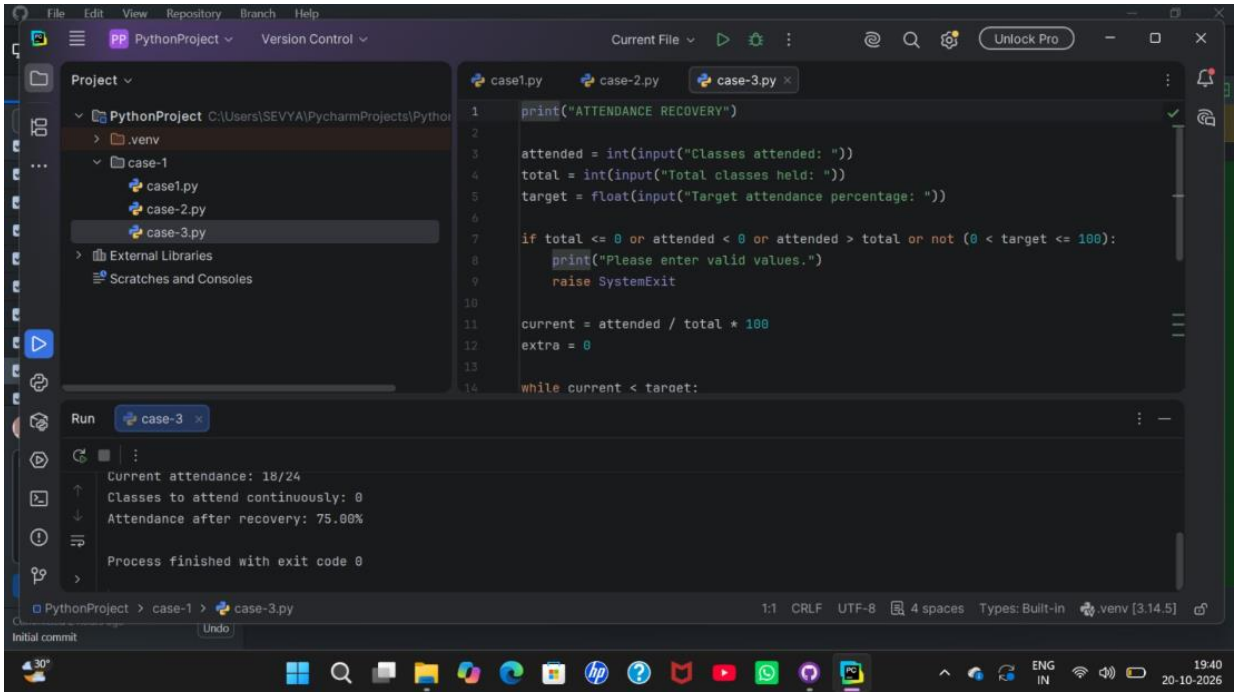
```
START
INPUT attended, total, target
IF total <= 0 OR attended < 0 OR attended > total OR target < 0 OR target > 100 THEN
    DISPLAY invalid values message
    STOP
END IF
current ← attended ÷ total × 100
extra ← 0
WHILE current < target
    attended ← attended + 1
    total ← total + 1
    extra ← extra + 1
    current ← attended ÷ total × 100
END WHILE
DISPLAY original/current attendance, extra, final percentage
STOP
```

Test Cases

Test	Input	Expected result
18 attended out of 24; target 75%	18, 24, 75	Current attendance = 75%; extra classes needed = 0
Attendance below target	Enter values where current percentage is below target	Program counts extra classes until target is reached
Invalid totals	Total classes = 0 or attended > total	Invalid-values message appears

Output Screenshot

The screenshot shows current attendance 18/24, 0 extra classes and 75.00% after recovery.


File Edit View Repository Branch Help
PythonProject
Version Control
Current File
Unlock Pro
Project
PythonProject C:\Users\SEVYA\PycharmProjects\Pytho
.venv
case-1
case1.py
case-2.py
case-3.py
External Libraries
Scratches and Consoles
case1.py
case-2.py
case-3.py
print("ATTENDANCE RECOVERY")
attended = int(input("Classes attended: "))
total = int(input("Total classes held: "))
target = float(input("Target attendance percentage: "))
if total <= 0 or attended < 0 or attended > total or not (0 < target <= 100):
print("Please enter valid values.")
raise SystemExit
current = attended / total * 100
extra = 0
while current < target:
Run
case-3
Current attendance: 18/24
Classes to attend continuously: 0
Attendance after recovery: 75.00%
Process finished with exit code 0
PythonProject > case-1 > case-3.py
1:1 CRLF UTF-8 4 spaces Types: Built-in .venv [3.14.5]
Undo
Initial commit
30°
ENG
IN
19:40
20-10-2026

5. ATM Transaction Menu

Aim: To simulate basic ATM operations such as checking balance, depositing money and withdrawing money.

Algorithm

32. Start with a balance of ₹5000.
33. Display the ATM menu: check balance, deposit, withdraw or exit.
34. Read the user's choice.
35. If choice is 1, display the current balance.
36. If choice is 2, read the deposit amount; add it if it is positive.
37. If choice is 3, read the withdrawal amount; withdraw only when the amount is valid and enough balance is available.
38. If choice is 4, exit the program.
39. For an invalid choice, display a message and show the menu again.

Flowchart (text form)

Start → Set balance = ₹5000 → Display menu → Read choice → Balance / Deposit / Withdraw / Exit → Update or display balance as needed → Exit chosen? → No: return to menu / Yes: Stop

Pseudocode

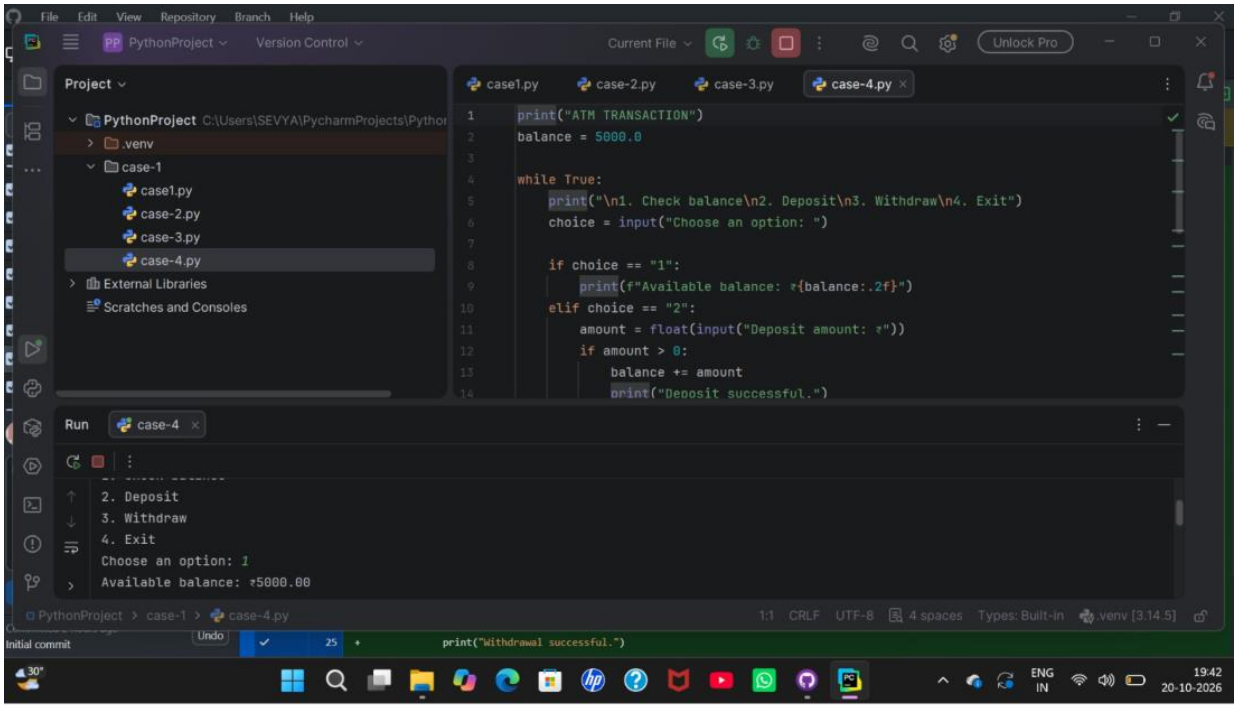
```
START
balance ← 5000
REPEAT
    DISPLAY menu: 1 Balance, 2 Deposit, 3 Withdraw, 4 Exit
    INPUT choice
    IF choice = 1 THEN DISPLAY balance
    ELSE IF choice = 2 THEN
        INPUT amount
        IF amount > 0 THEN balance ← balance + amount
        ELSE DISPLAY invalid amount
    ELSE IF choice = 3 THEN
        INPUT amount
        IF amount > 0 AND amount <= balance THEN
            balance ← balance - amount
        ELSE DISPLAY invalid amount or insufficient balance
    ELSE IF choice = 4 THEN EXIT LOOP
    ELSE DISPLAY invalid choice
UNTIL user exits
STOP
```

Test Cases

Test	Input	Expected result
Check balance	Choose 1	Available balance is ₹5000.00 initially
Deposit money	Choose 2 and enter a positive amount	Balance increases by the deposit
Withdraw within balance	Choose 3 and enter an amount not exceeding balance	Balance decreases by the withdrawal
Exit	Choose 4	Program ends

Output Screenshot

The screenshot shows the initial balance as ₹5000.00.


File Edit View Repository Branch Help
PythonProject
Version Control
Current File
Unlock Pro
Project
PythonProject C:\Users\SEVYA\PycharmProjects\Pytho
.venv
case-1
case1.py
case-2.py
case-3.py
case-4.py
External Libraries
Scratches and Consoles
case1.py
case-2.py
case-3.py
case-4.py
print("ATM TRANSACTION")
balance = 5000.0
while True:
print("\n1. Check balance\n2. Deposit\n3. Withdraw\n4. Exit")
choice = input("Choose an option: ")
if choice == "1":
print(f"Available balance: ₹{balance:.2f}")
elif choice == "2":
amount = float(input("Deposit amount: ₹"))
if amount > 0:
balance += amount
print("Deposit successful.")
Run
case-4
2. Deposit
3. Withdraw
4. Exit
Choose an option: 1
Available balance: ₹5000.00
PythonProject > case-1 > case-4.py
1:1 CRLF UTF-8 4 spaces Types: Built-in .venv (3.14.5)
Initial commit
Undo
print("Withdrawal successful.")
ENG
IN
19:42
20-10-2026

6. Performance Trend Analyzer

Aim: To calculate the average of test marks and identify the performance trend.

Algorithm

40. Start and input the number of tests.
41. If the number of tests is zero or negative, display a message and stop.
42. Create an empty list for marks.
43. Read and store the mark for each test.
44. Calculate the average using the sum of marks divided by the number of marks.
45. Compare the marks to decide whether performance is improving, declining or steady, according to the program's rules.
46. Display the average and trend.
47. Stop.

Flowchart (text form)

Start → Enter number of tests → Is count positive? → No: show message and stop / Yes: enter marks and store them → Calculate average → Compare marks to identify trend → Display average and trend → Stop

Pseudocode

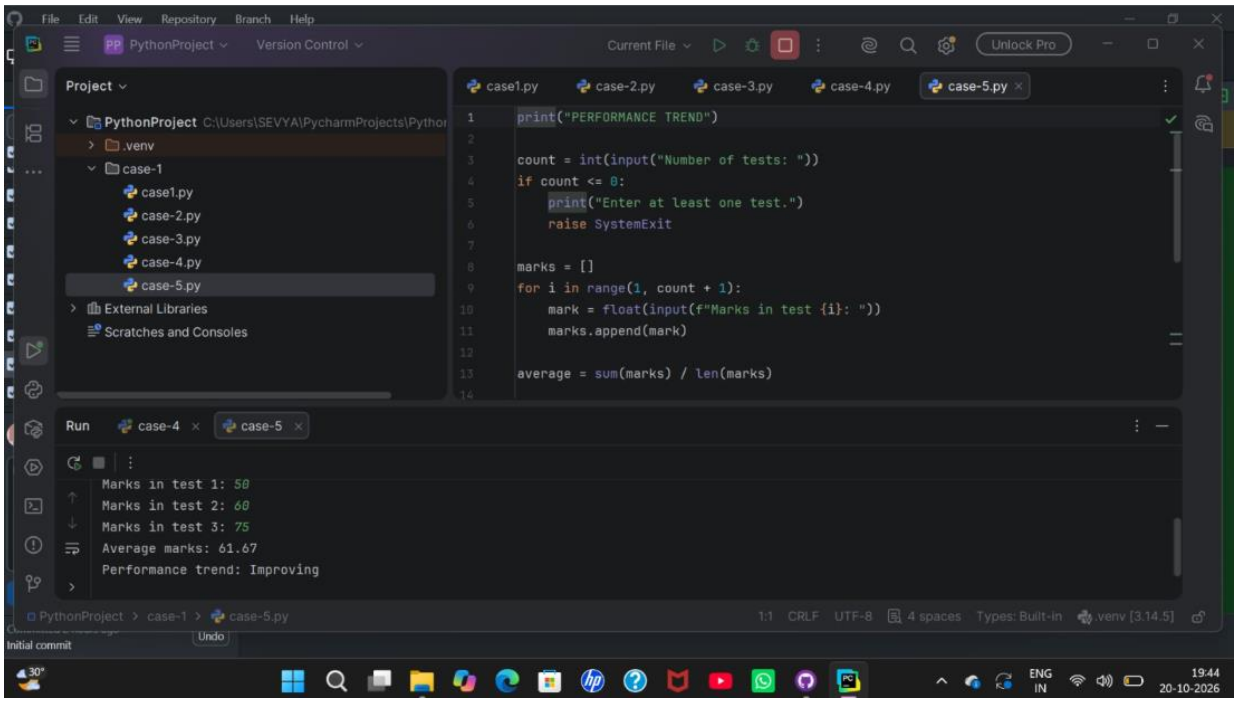
```
START
INPUT count
IF count <= 0 THEN DISPLAY message; STOP
marks ← empty list
FOR i ← 1 TO count
    INPUT mark
    ADD mark TO marks
END FOR
average ← SUM(marks) ÷ LENGTH(marks)
COMPARE marks using trend rules
DISPLAY average and performance trend
STOP
```

Test Cases

Test	Input	Expected result
Marks 50, 60, 75	Three tests	Average = 61.67; trend = Improving
No tests	Enter 0	Message asks for at least one test

Output Screenshot

The screenshot shows marks 50, 60 and 75, average 61.67 and trend Improving.


File Edit View Repository Branch Help
PythonProject
Version Control
Current File
Unlock Pro
Project
PythonProject C:\Users\SEVYA\PycharmProjects\Pythor
.venv
case-1
case1.py
case-2.py
case-3.py
case-4.py
case-5.py
External Libraries
Scratches and Consoles
case1.py
case-2.py
case-3.py
case-4.py
case-5.py
print("PERFORMANCE TREND")
count = int(input("Number of tests: "))
if count <= 0:
print("Enter at least one test.")
raise SystemExit
marks = []
for i in range(1, count + 1):
mark = float(input(f"Marks in test {i}: "))
marks.append(mark)
average = sum(marks) / len(marks)
Run
case-4
case-5
Marks in test 1: 50
Marks in test 2: 60
Marks in test 3: 75
Average marks: 61.67
Performance trend: Improving
PythonProject > case-1 > case-5.py
1:1 CRLF UTF-8 4 spaces Types: Built-in .venv [3.14.5]
Initial commit
Undo
ENG
IN
19:44
20-10-2026

7. Shopping Budget Checker

Aim: To calculate the total cost of items and check whether the total stays within a given budget.

Algorithm

48. Start and input the shopping budget.
49. Input the number of items.
50. Set total cost to 0.
51. For each item, read its cost.
52. If a cost is negative, display an error and stop.
53. Add each cost to the total.
54. Display the shopping total.
55. If total is less than or equal to budget, display the money left; otherwise display that the budget has been exceeded and show the extra amount.
56. Stop.

Flowchart (text form)

Start → Enter budget and number of items → Enter each item cost → Is cost non-negative? → No: show error / Yes: add cost to total → All items entered? → Display total → Is total within budget? → Display money left or amount over budget → Stop

Pseudocode

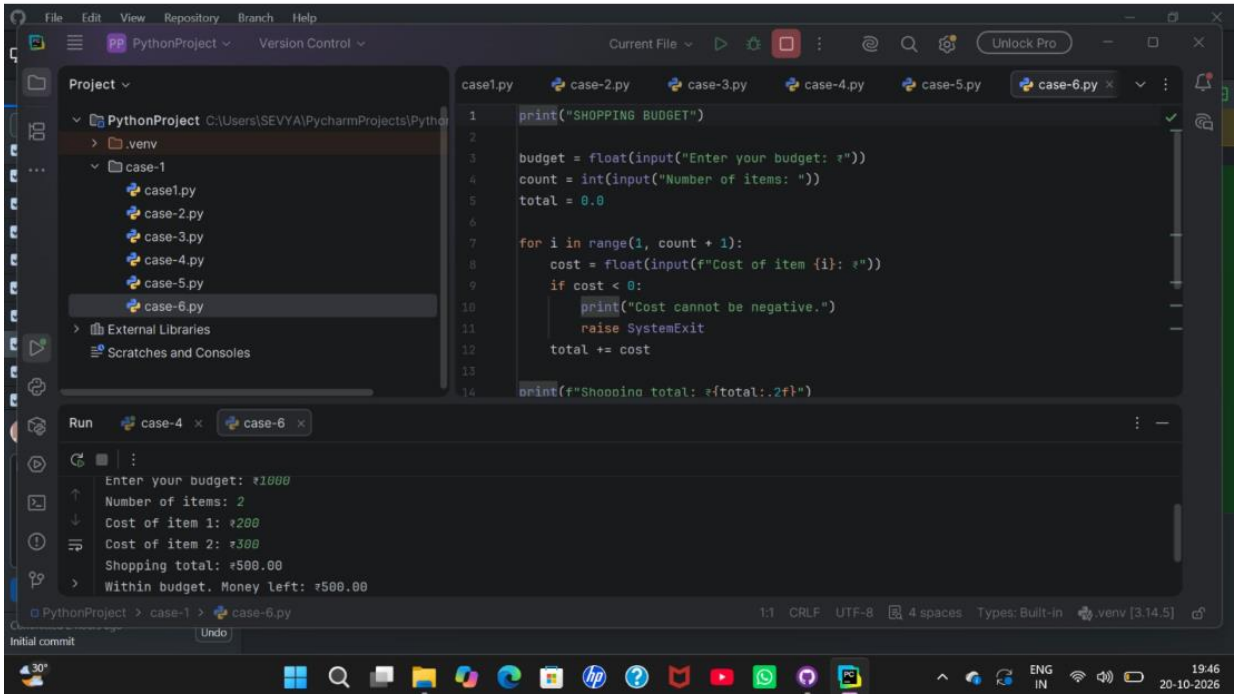
```
START
INPUT budget, count
total ← 0
FOR i ← 1 TO count
    INPUT cost
    IF cost < 0 THEN DISPLAY error; STOP
    total ← total + cost
END FOR
DISPLAY total
IF total <= budget THEN
    DISPLAY budget - total as money left
ELSE
    DISPLAY total - budget as amount over budget
END IF
STOP
```

Test Cases

Test	Input	Expected result
Budget ₹1000; costs ₹200 and ₹300	Two items	Total = ₹500.00; money left = ₹500.00
Total above budget	Enter costs whose total exceeds budget	Program reports that the budget is exceeded and shows the extra amount
Negative cost	Enter a cost below zero	Error message appears

Output Screenshot

The screenshot shows a ₹1000 budget, item costs ₹200 and ₹300, total ₹500.00 and ₹500.00 left.


File Edit View Repository Branch Help
PythonProject
Version Control
Current File
Unlock Pro
Project
PythonProject C:\Users\SEVYA\PycharmProjects\Pytho
.venv
case-1
case1.py
case-2.py
case-3.py
case-4.py
case-5.py
case-6.py
External Libraries
Scratches and Consoles
case1.py
case-2.py
case-3.py
case-4.py
case-5.py
case-6.py
print("SHOPPING BUDGET")
budget = float(input("Enter your budget: ₹"))
count = int(input("Number of items: "))
total = 0.0
for i in range(1, count + 1):
cost = float(input(f"Cost of item {i}: ₹"))
if cost < 0:
print("Cost cannot be negative.")
raise SystemExit
total += cost
print(f"Shopping total: ₹{total:.2f}")
Run
case-4
case-6
Enter your budget: ₹1000
Number of items: 2
Cost of item 1: ₹200
Cost of item 2: ₹300
Shopping total: ₹500.00
Within budget. Money left: ₹500.00
PythonProject > case-1 > case-6.py
1:1 CRLF UTF-8 4 spaces Types: Built-in .venv [3.14.5]
Initial commit
Undo
30°
ENG
IN
19:46
20-10-2026

8. Numeric Pattern Generator

Aim: To print a number pattern in which each row displays numbers from 1 up to that row number.

Algorithm

57. Start and input the number of rows.
58. If rows is zero or negative, display an error and stop.
59. Use an outer loop for each row from 1 to the requested number of rows.
60. Inside each row, use a loop to print numbers from 1 to the current row number on the same line.
61. Move to the next line after each row.
62. Stop.

Flowchart (text form)

Start → Enter number of rows → Is rows positive? → No: show error and stop / Yes: begin row loop → Print numbers from 1 to row number → Move to next line → More rows? → Yes: repeat / No: Stop

Pseudocode

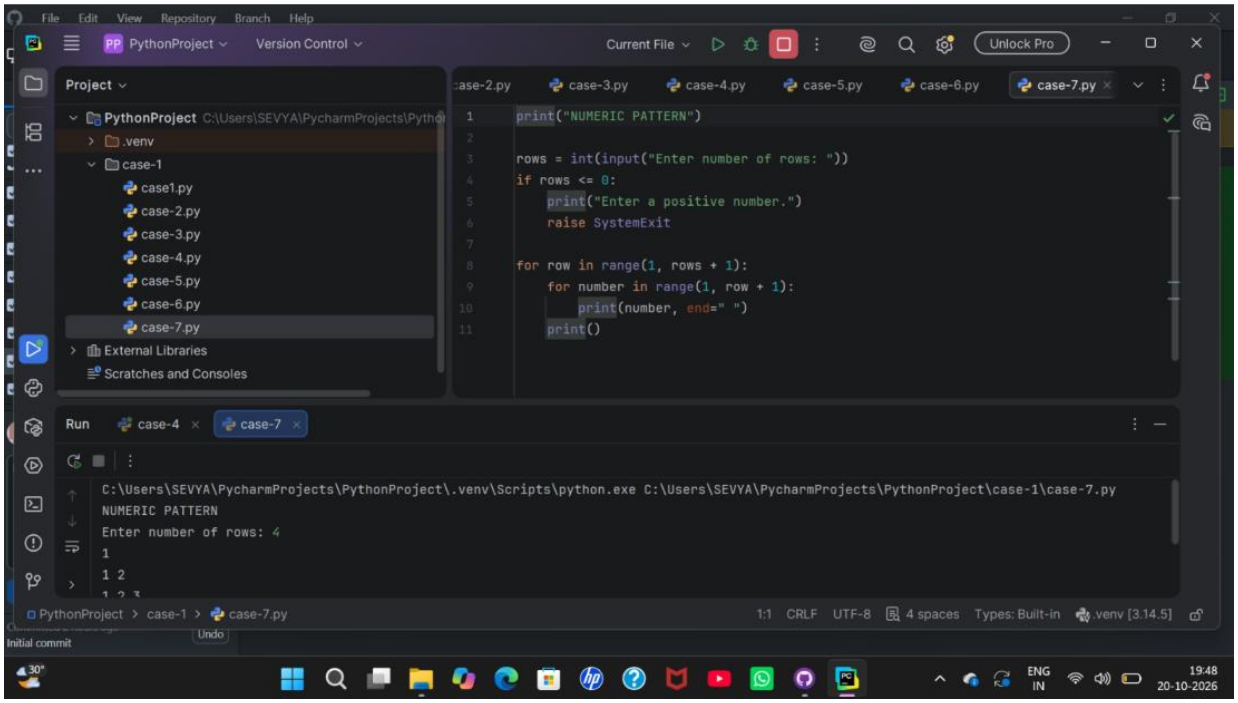
```
START
INPUT rows
IF rows <= 0 THEN DISPLAY error; STOP
FOR row ← 1 TO rows
    FOR number ← 1 TO row
        DISPLAY number WITHOUT newline
    END FOR
    DISPLAY newline
END FOR
STOP
```

Test Cases

Test	Input	Expected result
Rows = 4	Enter 4	Pattern rows are: 1; 1 2; 1 2 3; 1 2 3 4
Rows = 0	Enter 0	Positive-number message appears

Output Screenshot

The screenshot shows the pattern program running with 4 rows; the lower part of the output is not fully visible.


File Edit View Repository Branch Help
PythonProject
Version Control
Current File
Unlock Pro
Project
PythonProject C:\Users\SEVYA\PycharmProjects\Pytho
.venv
case-1
case1.py
case-2.py
case-3.py
case-4.py
case-5.py
case-6.py
case-7.py
External Libraries
Scratches and Consoles
case-2.py
case-3.py
case-4.py
case-5.py
case-6.py
case-7.py
print("NUMERIC PATTERN")
rows = int(input("Enter number of rows: "))
if rows <= 0:
print("Enter a positive number.")
raise SystemExit
for row in range(1, rows + 1):
for number in range(1, row + 1):
print(number, end=" ")
print()
Run
case-4
case-7
C:\Users\SEVYA\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\SEVYA\PycharmProjects\PythonProject\case-1\case-7.py
NUMERIC PATTERN
Enter number of rows: 4
1
1 2
PythonProject > case-1 > case-7.py
1:1 CRLF UTF-8 4 spaces Types: Built-in .venv [3.14.5]
Initial commit
Undo
ENG
IN
19:48
20-10-2026

9. Text Compression (Run-Length Encoding)

Aim: To shorten repeated characters by writing each character followed by the number of times it repeats consecutively.

Algorithm

63. Start and input a text string.
64. If the text is empty, display an empty-input message and stop.
65. Set current character to the first character and count to 1.
66. Read each remaining character one by one.
67. If it matches the current character, increase count.
68. Otherwise, add the current character and its count to the compressed string, then set the new character as current and reset count to 1.
69. After the loop, add the last character and its count.
70. Display the compressed text and stop.

Flowchart (text form)

Start → Enter text → Is text empty? → Yes: show message and stop / No: set current character and count = 1 → Read next character → Same as current? → Yes: increase count / No: append previous character and count; reset current and count → More characters? → Repeat → Append final character and count → Display compressed text → Stop

Pseudocode

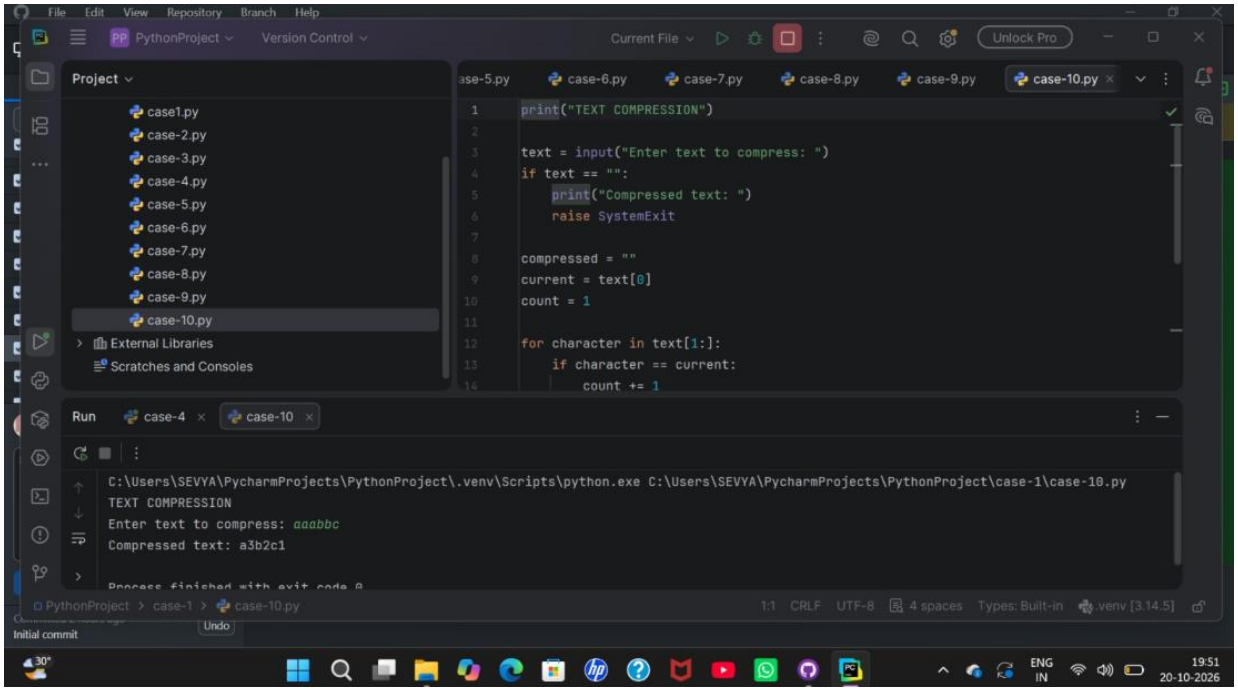
```
START
INPUT text
IF text is empty THEN DISPLAY message; STOP
compressed ← empty string
current ← first character of text
count ← 1
FOR each character in the rest of text
    IF character = current THEN
        count ← count + 1
    ELSE
        compressed ← compressed + current + count
        current ← character
        count ← 1
    END IF
END FOR
compressed ← compressed + current + count
DISPLAY compressed
STOP
```

Test Cases

Test	Input	Expected result
Text = aaabbc	Enter aaabbc	Compressed text = a3b2c1
Text = abcd	Enter abcd	Compressed text = a1b1c1d1
Empty text	Press Enter without typing	Empty-input message appears

Output Screenshot

The screenshot shows input aaabbc and compressed output a3b2c1. The last screenshot is a second view of the same program.


File Edit View Repository Branch Help
PythonProject
Version Control
Current File
Unlock Pro
Project
case1.py
case-2.py
case-3.py
case-4.py
case-5.py
case-6.py
case-7.py
case-8.py
case-9.py
case-10.py
External Libraries
Scratches and Consoles
case-5.py
case-6.py
case-7.py
case-8.py
case-9.py
case-10.py
print("TEXT COMPRESSION")
text = input("Enter text to compress: ")
if text == "":
print("Compressed text: ")
raise SystemExit
compressed = ""
current = text[0]
count = 1
for character in text[1:]:
if character == current:
count += 1
Run
case-4
case-10
C:\Users\SEVYA\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\SEVYA\PycharmProjects\PythonProject\case-1\case-10.py
TEXT COMPRESSION
Enter text to compress: aaabbc
Compressed text: a3b2c1
PythonProject > case-1 > case-10.py
1:1 CRLF UTF-8 4 spaces Types: Built-in .venv (3.14.5)
Initial commit
Undo
30°
ENG
IN
19:51
20-10-2026